

FlowOps

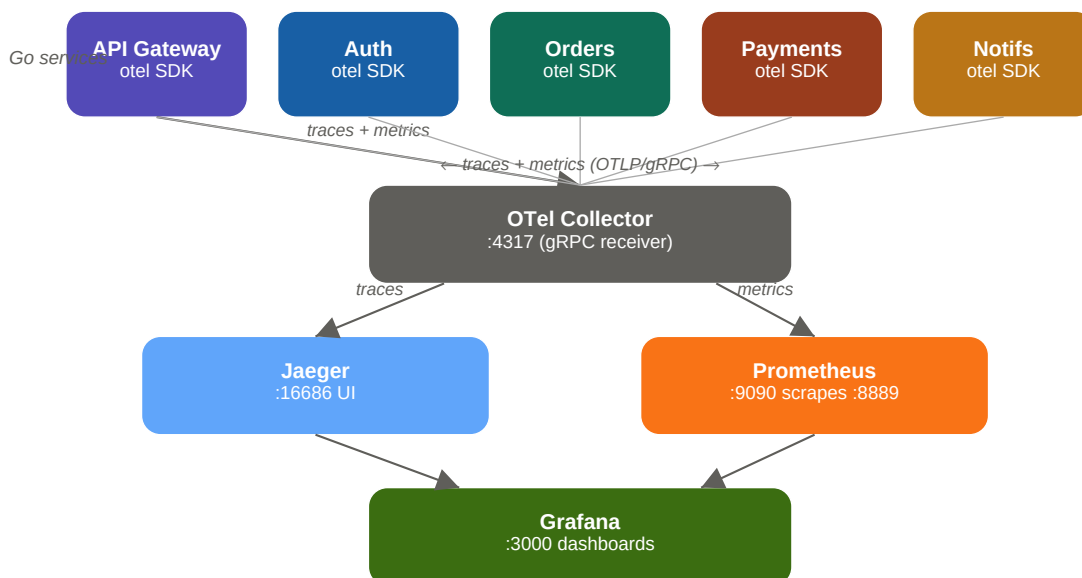
Phase 2 — OpenTelemetry Observability

Phase 2 adds full observability to the five Go services built in Phase 1. Every request that flows through the system gets traced end to end — you can see exactly which service was called, in what order, and how long each one took. This document covers the full stack: OTel Collector, Jaeger, Prometheus, Grafana, and how the Go services were instrumented.

What observability means

Without observability your services are **black boxes**. A request goes in, a response comes out, and you have no idea what happened in between. With observability every function call, every gRPC hop, every error gets recorded. You open Jaeger and see the entire journey of one request across all five services as a waterfall with millisecond timing.

The full observability stack



Go services emit traces and metrics → OTel Collector receives them → forwards traces to Jaeger, exposes metrics for Prometheus → Grafana visualises both.

1. Core concepts

OTLP — OpenTelemetry Protocol

OTLP is the standardised format OpenTelemetry uses to send data around. Think of it like JSON — a format everyone agreed on. Your Go services package their traces and metrics into OTLP format and send them to the collector. The collector speaks OTLP natively.

Traces and Spans

A **trace** is the full journey of one request across all services. A **span** is one unit of work inside that trace — for example, the Payments service processing a payment is one span. Spans nest inside each other to show the call hierarchy.

TLS: insecure: true

TLS is the encryption layer (same as HTTPS). Setting `insecure: true` disables encryption between services inside Docker. Fine for local development — everything runs on your machine. In production you would set this to false and provide certificates.

`context.Background()` vs `r.Context()`

`context.Background()` creates a brand new empty context with no trace information. Using it when making downstream gRPC calls **breaks the trace chain** — Jaeger sees disconnected spans instead of one linked trace. Always pass the incoming `ctx` (or `r.Context()` in HTTP handlers) so the trace propagates through every service.

Rule: never create `context.Background()` inside a handler function. Only use it at startup (initialising the tracer, connecting to services).

2. Docker Compose — observability stack

All four observability tools run as Docker containers defined in `docker-compose.yml` at the project root. No Dockerfiles needed — these are all official pre-built images.

SERVICE	IMAGE	PORTS
otel-collector	otel/opentelemetry-collector-contrib	4317:4317
jaeger	jaegertracing/all-in-one	16686:16686
prometheus	prom/prometheus	9090:9090
grafana	grafana/grafana	3000:3000

Full docker-compose.yml

```
services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:latest
    ports:
      - "4317:4317"
    volumes:
      - ./config/otel-collector-config.yaml:/etc/otel-collector-config.yaml
    command: ["--config=/etc/otel-collector-config.yaml"]

  jaeger:
    image: jaegertracing/all-in-one:latest
    ports:
      - "16686:16686"

  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./config/prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
```

Service names must be **lowercase** in Docker Compose. Capital letters cause DNS resolution issues inside the Docker network. These names become hostnames — `jaeger`, `otel-collector` etc. are how containers find each other.

3. Config files

config/otel-collector-config.yaml

This file tells the collector three things: how to **receive** data, where to **export** it, and how to **route** it (pipelines).

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317  # listen for traces/metrics from Go services

exporters:
  otlp/jaeger:  # forward traces to Jaeger
    endpoint: jaeger:4317
    tls:
      insecure: true

  prometheus:  # expose metrics for Prometheus to scrape
    endpoint: "0.0.0.0:8889"

service:
  pipelines:
    traces:
      receivers: [otlp]
      exporters: [otlp/jaeger]
    metrics:
      receivers: [otlp]
      exporters: [prometheus]
```

Receivers — how data comes IN. Listens on :4317 for OTLP over gRPC. **Exporters** — where data goes OUT. Jaeger for traces, port 8889 for Prometheus to scrape. **Pipelines** — wires receivers to exporters by data type.

config/prometheus.yml

Tells Prometheus where to scrape metrics from. Prometheus **pulls** metrics — it visits the target on a schedule. The OTel Collector exposes metrics on port 8889.

```
scrape_configs:
  - job_name: 'otel-collector'
    scrape_interval: 5s
    static_configs:
      - targets: ['otel-collector:8889']
```

otel-collector:8889 works because inside Docker, containers reach each other by service name. otel-collector resolves to the collector container's IP automatically.

4. shared/telemetry.go — the tracer initialiser

Instead of writing OTel setup code in all five services, you write it once in `shared/telemetry.go` and every service imports it. One function: `InitTracer(serviceName string)`.

```
package shared

import (
    "context"
    "log"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc"
    "go.opentelemetry.io/otel/propagation"
    "go.opentelemetry.io/otel/sdk/resource"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    semconv "go.opentelemetry.io/otel/semconv/v1.4.0"
    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
)

func InitTracer(serviceName string) func() {
    ctx := context.Background()

    // connect to OTel Collector
    conn, _ := grpc.NewClient("localhost:4317",
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )

    // create exporter (packages traces into OTLP and sends via conn)
    exporter, _ := otlptracegrpc.New(ctx, otlptracegrpc.WithGRPCConn(conn))

    // label this service's traces with its name
    res := resource.NewWithAttributes(
        semconv.SchemaURL,
        semconv.ServiceNameKey.String(serviceName),
    )

    // create tracer provider – the factory that creates tracers
    tp := sdktrace.NewTracerProvider(
        sdktrace.WithBatcher(exporter),
        sdktrace.WithResource(res),
    )

    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.TraceContext{})

    // return cleanup function – flushes unsent traces on shutdown
    return func() {
        tp.Shutdown(ctx)
    }
}
```

What each part does:

- **conn** — TCP connection to OTel Collector on :4317
- **exporter** — packages traces into OTLP format and sends via conn
- **resource** — labels traces with the service name so Jaeger knows which service sent them
- **TracerProvider** — global factory, set once, used everywhere in the service
- **TextMapPropagator** — enables trace context to flow between services via headers
- **returned func()** — call with defer in main() to flush traces on shutdown

5. Adding observability to each service

Step 1 — Call `InitTracer` at the top of `main()`

Two lines in every service's `main()`. Only the service name string changes:

```
func main() {
    shutdown := shared.InitTracer("payments") // "auth", "orders", etc.
    defer shutdown()                          // flush traces on exit

    // ... rest of main
}
```

Step 2 — Add gRPC server instrumentation

For gRPC servers (Auth, Orders, Payments, Notifications), pass `otelgrpc.NewServerHandler()` when creating the gRPC server. This auto-creates spans for every incoming gRPC call — no manual span code needed in every handler.

```
grpcServer := grpc.NewServer(
    grpc.StatsHandler(otelgrpc.NewServerHandler()),
)
```

Step 3 — Add gRPC client instrumentation

For services that make outgoing gRPC calls (Orders calls Payments + Notifications, API Gateway calls Auth + Orders), add `otelgrpc.NewClientHandler()` to each client connection:

```
paymentsConn, _ := grpc.NewClient("localhost:50053",
    grpc.WithTransportCredentials(insecure.NewCredentials()),
    grpc.WithStatsHandler(otelgrpc.NewClientHandler()), // <-- add this
)
```

Step 4 — Add manual spans inside handlers (optional but recommended)

The server/client handlers auto-create spans at the gRPC level. You can also create custom spans inside your handler functions for more granular tracing:

```
func (s *PaymentServer) ProcessPayment(ctx context.Context, req *pb.ProcessPaymentRequest) (*pb.ProcessPaymentResponse, error) {
    tracer := otel.Tracer("payments")
    _, span := tracer.Start(ctx, "ProcessPayment")
    defer span.End()

    // your logic here
    return &pb.ProcessPaymentResponse{Success: true}, nil
}
```

Important: pass the same `ctx` you received into `tracer.Start()`. This links the new span to the parent span, keeping the trace chain intact.

6. What you see in Jaeger

After sending a request and opening `http://localhost:16686`:

- Select **api-gateway** from the Service dropdown
- Click **Find Traces**
- Each row is one complete request — click it to see the waterfall

What a trace looks like:

```
api-gateway: orders.OrderService/CreateOrder [9 spans, 2.13ms]
  ■■■ api-gateway (1 span)
    ■ ■■■ outgoing call to auth
      ■■■ auth (2 spans)
        ■ ■■■ Validate token
      ■■■ orders (4 spans)
        ■ ■■■ CreateOrder handler
        ■ ■■■ outgoing call to payments
        ■ ■■■ outgoing call to notifications
      ■■■ payments (2 spans)
        ■ ■■■ ProcessPayment handler
      ■■■ notifications (2 spans)
        ■■■ SendNotifications handler
```

Each bar's width represents how long that span took. You can instantly see which service is the bottleneck. Click any span to see its tags, logs, and exact timestamps.

Why traces weren't showing initially

Symptom	Root Cause	Fix
No services in Jaeger dropdown	No spans created yet	Add span code to handlers
Only payments showing	Other services had no requests	Send a full order request
Disconnected spans in Jaeger	Using <code>context.Background()</code>	Use <code>r.Context()</code> in gateway
Services not in dropdown	<code>InitTracer</code> not called	Add <code>shared.InitTracer()</code> to <code>main()</code>

7. Ports reference

All ports used across Phase 1 and Phase 2:

Service	Port	Protocol	Access
API Gateway	:8080	HTTP	Your curl requests
Auth	:50051	gRPC	Internal only
Orders	:50052	gRPC	Internal only
Payments	:50053	gRPC	Internal only
Notifications	:50054	gRPC	Internal only
OTel Collector	:4317	gRPC	Go services send traces here
OTel Metrics	:8889	HTTP	Prometheus scrapes here
Jaeger UI	:16686	HTTP	localhost:16686 in browser
Prometheus UI	:9090	HTTP	localhost:9090 in browser
Grafana UI	:3000	HTTP	localhost:3000 in browser

8. Commands reference

Install OTel packages (run once at project root):

```
go get go.opentelemetry.io/otel
go get go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc
go get go.opentelemetry.io/otel/sdk/trace
go get go.opentelemetry.io/otel/propagation
go get go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc
```

Start observability stack:

```
docker compose up -d
```

Start all services (each in separate terminal):

```
go run services/payments/main.go
go run services/notifications/main.go
go run services/auth/main.go
go run services/orders/main.go
go run services/api-gateway/main.go
```

Send a test request (PowerShell):

```
curl.exe -X POST http://localhost:8080/order -H "Content-Type: application/json" -d "{\"token\":\"\"}"

# Expected
{"order_id":"order-1780816813","success":true}
```

Check everything is running:

```
netstat -ano | findstr "50051 50052 50053 50054 8080"
```

Phase 2 complete. Five services instrumented, full distributed tracing in Jaeger, metrics in Prometheus, Grafana ready for dashboards.

Next: Phase 3 — Kubernetes. Write Helm charts, deploy to a local kind cluster.